

Universidad Privada Domingo Savio

Tecnologias Web II

Entrega 9A

Modulo de Asistencias: Registro Docente

FASE 4: Horarios, Asistencias y Configuracion de Evaluacion

Abril 2026

Objetivos de Aprendizaje

- Diseñar un schema Zod para registro masivo de asistencias (array de registros en una sola peticion).
- Implementar API Routes que soporten operaciones bulk INSERT con transaccion atomica.
- Construir un componente interactivo de 'pasar lista' que muestre todos los estudiantes inscritos en una materia.
- Crear una vista de historial de asistencias con filtros por fecha y estado.
- Aplicar el patron de consulta en 2 pasos para obtener datos contextuales (inscripciones + perfil del estudiante).

Prerrequisitos

- ✓ Entrega 8A completada: tabla asistencias creada con RLS habilitado.
- ✓ Entrega 8B completada: modulo de horarios funcional.
- ✓ Al menos 1 materia con docente asignado y 2+ estudiantes inscritos en Supabase.
- ✓ Trigger trigger_updated_at_asistencias activo en la tabla asistencias.

Teoria: Registro Masivo vs Individual

A diferencia de las tablas anteriores donde cada registro se crea individualmente (un horario, una inscripcion), la asistencia tiene un patron de uso diferente: el docente pasa lista a TODOS los estudiantes de una materia en una sola sesion. Esto introduce el concepto de operacion masiva (bulk operation).

Individual vs Masivo

Aspecto	INSERT Individual	INSERT Masivo (bulk)
Peticiones HTTP	1 por cada estudiante	1 sola con array de registros
Latencia total	$N * \text{latencia_red}$	$1 * \text{latencia_red}$
Atomicidad	Cada insert es independiente	Todos se guardan o ninguno
UX para el docente	Click por cada estudiante	Marca todos y guarda con 1 click
Ejemplo	POST { inscripcion_id, estado }	POST { registros: [{...}, {...}] }

NOTA: Por que masivo?

Si una materia tiene 30 estudiantes inscritos, el enfoque individual haria 30 peticiones HTTP al guardar. Con el enfoque masivo, una sola peticion envia los 30 registros y Supabase los inserta en una transaccion atomica con `.insert(array)`.

Flujo del Modulo de Asistencias (Docente)



```
| (presente/ausente) | | | | |
|-- Click 'Guardar' ---->| |
| | |
| | -- Zod.safeParse() ----->| |
| | -- POST /api/docente/ -->| |
| | asistencias | |
| | | | -- UPSERT[] ----->| |
| | | | <-- ok ----->| |
| | | |
| | <-- 201 + toast ----->| |
| <-- Confirmacion ----->| |
```

TIP: UPSERT en lugar de INSERT

Usamos UPSERT (INSERT ... ON CONFLICT UPDATE) para que el docente pueda volver a una fecha y modificar los estados sin errores de duplicado. Si ya existe un registro para (inscripcion_id, fecha), se actualiza; si no existe, se crea.

Paso 1: Schema de Validacion Zod

Crea el schema de validacion para asistencias. La diferencia clave con el schema de horarios es que aqui validamos un ARRAY de registros:

TypeScript - lib/validations/asistencia.ts (crear archivo nuevo)

```
import { z } from "zod";

// Schema para un registro individual de asistencia
export const asistenciaRegistroSchema = z.object({
  inscripcion_id: z
    .string()
    .uuid("ID de inscripcion invalido"),

  estado: z.enum(
    ["presente", "ausente", "justificado", "tardanza"],
    {
      errorMap: () => ({ message: "Estado de asistencia invalido" })
    }
  ),

  observaciones: z
    .string()
    .max(500, "Maximo 500 caracteres")
    .optional()
    .or(z.literal("")),
});

// Schema para el envio masivo (pasar lista completa)
export const asistenciaMasivaSchema = z.object({
  materia_id: z
    .string()
    .uuid("ID de materia invalido"),

  fecha: z
    .string()
    .regex(
      /^\\d{4}-\\d{2}-\\d{2}$/,
      "Formato de fecha invalido (YYYY-MM-DD)"
    )
    .refine((val) => {
      const fecha = new Date(val);
      const hoy = new Date();
      hoy.setHours(23, 59, 59, 999);
      return fecha <= hoy;
    }, "No se puede registrar asistencia para fechas futuras"),

  registros: z
    .array(asistenciaRegistroSchema)
```

```

    .min(1, "Debe incluir al menos un registro")
    .max(100, "Maximo 100 registros por envio"),
  });

export type AsistenciaRegistro = z.infer<typeof asistenciaRegistroSchema>;
export type AsistenciaMasiva = z.infer<typeof asistenciaMasivaSchema>;

// Constantes para los selects del formulario
export const ESTADOS_ASISTENCIA = [
  { value: "presente", label: "Presente", color: "bg-green-100 text-green-800" },
  { value: "ausente", label: "Ausente", color: "bg-red-100 text-red-800" },
  { value: "justificado", label: "Justificado", color: "bg-yellow-100 text-yellow-800" },
  { value: "tardanza", label: "Tardanza", color: "bg-orange-100 text-orange-800" },
];

```

Anatomia del Schema Masivo

Campo	Tipo	Validacion	Proposito
materia_id	string.uuid	Requerido	Identifica la materia (para verificar que el docente es dueño)
fecha	string.regex + refine	YYYY-MM-DD, no futuro	Fecha de la sesion de clase
registros	array(registro)	min 1, max 100	Array con el estado de cada estudiante
registros[].inscripcion_id	string.uuid	Requerido	Vincula al estudiante inscrito
registros[].estado	enum 4 valores	Requerido	presente, ausente, justificado, tardanza
registros[].observaciones	string.max(500)	Opcional	Nota del docente sobre el estudiante

ADVERTENCIA: Validacion de fecha futura

El `.refine()` impide que el docente registre asistencia para mañana o fechas futuras. Sin embargo, SI permite registrar asistencia retroactiva (días pasados) porque es común que un docente olvide pasar lista y lo haga al día siguiente.

Paso 2: API Route Docente - GET Asistencias

El endpoint GET tiene dos modos: (1) listar estudiantes inscritos para pasar lista en una fecha especifica, y (2) obtener el historial de asistencias de una materia. Los query params determinan el modo.

TypeScript - app/api/docente/asistencias/route.ts (crear archivo nuevo)

```
import { createClient } from "@lib/supabase/server";
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  const supabase = await createClient();

  // Autenticacion
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Query params
  const { searchParams } = new URL(request.url);
  const materia_id = searchParams.get("materia_id");
  const fecha = searchParams.get("fecha");
  const modo = searchParams.get("modo") || "lista";

  if (!materia_id) {
    return NextResponse.json(
      { error: "materia_id es requerido" },
      { status: 400 }
    );
  }

  // Verificar que la materia pertenece al docente
  const { data: materia } = await supabase
    .from("materias")
    .select("id, nombre")
    .eq("id", materia_id)
    .eq("docente_id", user.id)
    .single();

  if (!materia) {
    return NextResponse.json(
      { error: "No tienes acceso a esta materia" },
      { status: 403 }
    );
  }
}
```

```

// — MODO LISTA: para pasar lista en una fecha —
if (modo === "lista" && fecha) {
  // Obtener inscripciones activas con perfil del estudiante
  const { data: inscripciones, error: inscError } = await supabase
    .from("inscripciones")
    .select(`
      id,
      estudiante_id,
      usuarios_perfil!inscripciones_estudiante_id_fkey (
        id, nombre, apellido, email
      )
    `)
    .eq("materia_id", materia_id)
    .eq("estado", "inscrito")
    .order("created_at");

  if (inscError) {
    return NextResponse.json(
      { error: inscError.message },
      { status: 500 }
    );
  }

  // Obtener asistencias existentes para esa fecha
  const inscripcionIds = inscripciones?.map((i) => i.id) || [];

  let asistenciasExistentes: any[] = [];
  if (inscripcionIds.length > 0) {
    const { data } = await supabase
      .from("asistencias")
      .select("*")
      .in("inscripcion_id", inscripcionIds)
      .eq("fecha", fecha);

    asistenciasExistentes = data || [];
  }

  // Combinar: inscripcion + asistencia existente (si hay)
  const listaCompleta = inscripciones?.map((insc) => {
    const asistencia = asistenciasExistentes.find(
      (a) => a.inscripcion_id === insc.id
    );
    return {
      inscripcion_id: insc.id,
      estudiante: insc.usuarios_perfil,
      asistencia_existente: asistencia || null,
    };
  });

  return NextResponse.json({
    materia,

```



```

    fecha,
    estudiantes: listaCompleta,
  });
}

// — MODO HISTORIAL: todas las asistencias —
if (modo === "historial") {
  const { data: inscripciones } = await supabase
    .from("inscripciones")
    .select("id")
    .eq("materia_id", materia_id)
    .eq("estado", "inscrito");

  const inscIds = inscripciones?.map((i) => i.id) || [];

  if (inscIds.length === 0) {
    return NextResponse.json({ historial: [], fechas: [] });
  }

  const { data: historial, error: histError } = await supabase
    .from("asistencias")
    .select(`
      *,
      inscripciones!inner (
        id,
        estudiante_id,
        usuarios_perfil!inscripciones_estudiante_id_fkey (
          nombre, apellido
        )
      )
    `)
    .in("inscripcion_id", inscIds)
    .order("fecha", { ascending: false })
    .order("created_at");

  if (histError) {
    return NextResponse.json(
      { error: histError.message },
      { status: 500 }
    );
  }

  // Extraer fechas unicas para el filtro
  const fechasUnicas = [
    ...new Set(historial?.map((h) => h.fecha) || []),
  ];

  return NextResponse.json({
    historial,
    fechas: fechasUnicas,
  });
}

```

```
    });  
  }  
  
  return NextResponse.json(  
    { error: "Parametros insuficientes: envíe fecha con modo=lista o modo=historial" },  
    { status: 400 }  
  );  
}
```

NOTA: Doble consulta en modo lista

En modo 'lista', hacemos 2 consultas: primero inscripciones (para obtener todos los estudiantes), luego asistencias existentes. Esto permite mostrar el formulario con los estados pre-cargados si el docente ya paso lista ese día, habilitando la corrección de errores.

Paso 3: API Route Docente - POST Masivo

El POST recibe el array completo de registros y los inserta/actualiza en una sola operacion con UPSERT. Esto es la operacion mas critica del modulo.

TypeScript - Agregar POST al mismo archivo route.ts del Paso 2

```
// — POST: Registrar asistencia masiva —
export async function POST(request: NextRequest) {
  const supabase = await createClient();

  // Autenticacion
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Importar validacion
  const { asistenciaMasivaSchema } = await import(
    "@/lib/validations/asistencia"
  );

  // Parsear y validar body
  const body = await request.json();
  const resultado = asistenciaMasivaSchema.safeParse(body);

  if (!resultado.success) {
    return NextResponse.json(
      { error: resultado.error.errors[0].message },
      { status: 400 }
    );
  }

  const { materia_id, fecha, registros } = resultado.data;

  // Verificar que la materia pertenece al docente
  const { data: materia } = await supabase
    .from("materias")
    .select("id")
    .eq("id", materia_id)
    .eq("docente_id", user.id)
    .single();

  if (!materia) {
    return NextResponse.json(
      { error: "No tienes acceso a esta materia" },
      { status: 403 }
    );
  }
}
```

```

}

// Verificar que todas las inscripciones pertenecen a la materia
const inscripcionIds = registros.map((r) => r.inscripcion_id);

const { data: inscripciones } = await supabase
  .from("inscripciones")
  .select("id")
  .eq("materia_id", materia_id)
  .in("id", inscripcionIds);

if (!inscripciones || inscripciones.length !== inscripcionIds.length) {
  return NextResponse.json(
    { error: "Algunas inscripciones no pertenecen a esta materia" },
    { status: 400 }
  );
}

// Preparar registros para UPSERT
const datosUpsert = registros.map((r) => ({
  inscripcion_id: r.inscripcion_id,
  fecha,
  estado: r.estado,
  observaciones: r.observaciones || null,
  registrado_por: user.id,
})));

// UPSERT masivo
const { data: resultados, error } = await supabase
  .from("asistencias")
  .upsert(datosUpsert, {
    onConflict: "inscripcion_id,fecha",
    ignoreDuplicates: false,
  })
  .select();

if (error) {
  return NextResponse.json(
    { error: error.message },
    { status: 500 }
  );
}

return NextResponse.json(
  {
    mensaje: `Asistencia registrada: ${resultados?.length || 0} registros`,
    registros: resultados,
  },
  { status: 201 }
);

```

}

Como funciona el UPSERT de Supabase

Parametro	Valor	Efecto
onConflict	"inscripcion_id,fecha"	Define la columna UNIQUE que detecta duplicados
ignoreDuplicates	false	Si hay duplicado, ACTUALIZA en vez de ignorar
Primer envio	INSERT normal	Crea registros nuevos para cada estudiante
Segundo envio misma fecha	UPDATE automatico	Actualiza estado/observaciones sin error

IMPORTANTE: Verificacion de integridad

Antes del UPSERT, verificamos que TODAS las inscripcion_id enviadas pertenecen a la materia del docente. Sin esta validacion, un docente podria enviar inscripcion_id de otra materia y registrar asistencia donde no deberia.

Paso 4: API Route Docente - PUT Individual

Ademas del registro masivo, permitimos la edicion individual de un registro de asistencia (para corregir un estado especifico desde el historial).

```
TypeScript - app/api/docente/asistencias/[id]/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const body = await request.json();

  // Validar que el estado es valido
  const estadosValidos = ["presente", "ausente", "justificado", "tardanza"];
  if (!estadosValidos.includes(body.estado)) {
    return NextResponse.json(
      { error: "Estado de asistencia invalido" },
      { status: 400 }
    );
  }

  // Verificar que el registro existe y pertenece al docente
  const { data: asistencia } = await supabase
    .from("asistencias")
    .select(`
      id,
      inscripciones!inner (
        materia_id,
        materias!inner ( docente_id )
      )
    `)
    .eq("id", id)
    .single();

  if (!asistencia) {
    return NextResponse.json(
```

```

    { error: "Registro no encontrado" },
    { status: 404 }
  );
}

// Verificar propiedad via chain: asistencia -> inscripcion -> materia -> docente
const docente_id = (asistencia as any).inscripciones?.materias?.docente_id;
if (docente_id !== user.id) {
  return NextResponse.json(
    { error: "No tienes permiso para editar este registro" },
    { status: 403 }
  );
}

// Actualizar
const { data: actualizado, error } = await supabase
  .from("asistencias")
  .update({
    estado: body.estado,
    observaciones: body.observaciones || null,
  })
  .eq("id", id)
  .select()
  .single();

if (error) {
  return NextResponse.json({ error: error.message }, { status: 500 });
}

return NextResponse.json(actualizado);
}

```

NOTA: Cadena de verificación de propiedad

Para verificar que el docente puede editar un registro de asistencia, seguimos la cadena: asistencia -> inscripcion -> materia -> docente_id. Esto es mas seguro que confiar solo en RLS porque agrega validacion explicita a nivel de aplicacion.

Paso 5: Componente Pasar Lista

Este es el componente central del modulo. Muestra una tabla con todos los estudiantes inscritos y permite al docente seleccionar el estado de cada uno. Al guardar, envia todos los registros en una sola peticion.

TypeScript - components/dashboard/docente/pasar-lista.tsx (Parte 1: Imports y Estado)

```
"use client";

import { useEffect, useState, useCallback } from "react";
import { toast } from "sonner";
import { CalendarDays, Save, Users, CheckCircle } from "lucide-react";

import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Textarea } from "@components/ui/textarea";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";

import { ESTADOS_ASISTENCIA } from "@lib/validations/asistencia";

interface Estudiante {
  inscripcion_id: string;
  estudiante: {
    id: string;
    nombre: string;
    apellido: string;
    email: string;
  };
  asistencia_existente: {
    id: string;
    estado: string;
    observaciones: string | null;
  } | null;
}

interface RegistroLocal {
  inscripcion_id: string;
  estado: string;
}
```



```

    observaciones: string;
}

interface Materia {
    id: string;
    nombre: string;
}

```

TypeScript - pasar-lista.tsx (Parte 2: Componente Principal)

```

export default function PasarLista() {
    // Estado de seleccion
    const [materias, setMaterias] = useState<Materia[]>([]);
    const [materiaSeleccionada, setMateriaSeleccionada] = useState("");
    const [fecha, setFecha] = useState(
        new Date().toISOString().split("T")[0]
    );

    // Estado de la lista
    const [estudiantes, setEstudiantes] = useState<Estudiante[]>([]);
    const [registros, setRegistros] = useState<RegistroLocal[]>([]);
    const [cargando, setCargando] = useState(false);
    const [guardando, setGuardando] = useState(false);
    const [listaCargada, setListaCargada] = useState(false);

    // Cargar materias del docente al montar
    useEffect(() => {
        async function cargarMaterias() {
            const res = await fetch("/api/docente/materias");
            if (res.ok) {
                const data = await res.json();
                setMaterias(data);
            }
        }
        cargarMaterias();
    }, []);

    // Cargar lista cuando cambia materia o fecha
    const cargarLista = useCallback(async () => {
        if (!materiaSeleccionada || !fecha) return;

        setCargando(true);
        setListaCargada(false);

        try {
            const params = new URLSearchParams({
                materia_id: materiaSeleccionada,
                fecha,
                modo: "lista",
            });

```

```

const res = await fetch(`/api/docente/asistencias?${params}`);

if (!res.ok) {
  const err = await res.json();
  toast.error(err.error);
  return;
}

const data = await res.json();
setEstudiantes(data.estudiantes);

// Inicializar registros locales con estado existente o "presente"
const registrosIniciales = data.estudiantes.map(
  (est: Estudiante) => ({
    inscripcion_id: est.inscripcion_id,
    estado: est.asistencia_existente?.estado || "presente",
    observaciones: est.asistencia_existente?.observaciones || "",
  })
);
setRegistros(registrosIniciales);
setListaCargada(true);
} catch {
  toast.error("Error al cargar la lista");
} finally {
  setCargando(false);
}
}, [materiaSeleccionada, fecha]);

```

TypeScript - pasar-lista.tsx (Parte 3: Guardar y Helpers)

```

// — Guardar asistencia masiva —
async function guardarAsistencia() {
  setGuardando(true);
  try {
    const res = await fetch("/api/docente/asistencias", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        materia_id: materiaSeleccionada,
        fecha,
        registros,
      }),
    });
  }

  if (!res.ok) {
    const err = await res.json();
    toast.error(err.error);
    return;
  }
}

```

```

    const data = await res.json();
    toast.success(data.mensaje);
  } catch {
    toast.error("Error al guardar la asistencia");
  } finally {
    setGuardando(false);
  }
}

// — Actualizar estado de un estudiante —
function actualizarEstado(inscripcionId: string, estado: string) {
  setRegistros((prev) =>
    prev.map((r) =>
      r.inscripcion_id === inscripcionId
        ? { ...r, estado }
        : r
    )
  );
}

// — Actualizar observacion —
function actualizarObservacion(inscripcionId: string, obs: string) {
  setRegistros((prev) =>
    prev.map((r) =>
      r.inscripcion_id === inscripcionId
        ? { ...r, observaciones: obs }
        : r
    )
  );
}

// — Marcar todos con un estado —
function marcarTodos(estado: string) {
  setRegistros((prev) =>
    prev.map((r) => ({ ...r, estado })))
  );
}

// — Contadores de resumen —
const contadores = ESTADOS_ASISTENCIA.map((ea) => ({
  ...ea,
  count: registros.filter((r) => r.estado === ea.value).length,
})));

```

TypeScript - pasar-lista.tsx (Parte 4: Render)

```

return (
  <div className="space-y-6">
    {/* Selectores de materia y fecha */}

```

```

<Card>
  <CardContent className="pt-6">
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4 items-end">
      <div className="grid gap-2">
        <Label>Materia</Label>
        <Select
          value={materiaSeleccionada}
          onChange={setMateriaSeleccionada}
        >
          <SelectTrigger>
            <SelectValue placeholder="Seleccionar materia" />
          </SelectTrigger>
          <SelectContent>
            {materias.map((m) => (
              <SelectItem key={m.id} value={m.id}>
                {m.nombre}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>

      <div className="grid gap-2">
        <Label>Fecha</Label>
        <Input
          type="date"
          value={fecha}
          max={new Date().toISOString().split("T")[0]}
          onChange={(e) => setFecha(e.target.value)}
        />
      </div>

      <Button onClick={cargarLista} disabled={cargando}>
        <CalendarDays className="h-4 w-4 mr-2" />
        {cargando ? "Cargando..." : "Cargar Lista"}
      </Button>
    </div>
  </CardContent>
</Card>

{/* Tabla de asistencia */}
{listaCargada && (
  <Card>
    <CardHeader>
      <div className="flex items-center justify-between">
        <CardTitle className="flex items-center gap-2">
          <Users className="h-5 w-5" />
          Lista de Asistencia ({estudiantes.length} estudiantes)
        </CardTitle>
        <div className="flex gap-2">
          {ESTADOS_ASISTENCIA.slice(0, 2).map((ea) => (
            <Button

```

```

        key={ea.value}
        variant="outline"
        size="sm"
        onClick={() => marcarTodos(ea.value)}
      >
        Todos {ea.label}
      </Button>
    )}}
  </div>
</div>

{/* Resumen de contadores */}
<div className="flex gap-3 mt-2">
  {contadores.map((c) => (
    <Badge key={c.value} className={c.color}>
      {c.label}: {c.count}
    </Badge>
  ))}
</div>
</CardHeader>

<CardContent>
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead className="w-[40px]">#</TableHead>
        <TableHead>Estudiante</TableHead>
        <TableHead>Estado</TableHead>
        <TableHead>Observaciones</TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {estudiantes.map((est, index) => {
        const registro = registros.find(
          (r) => r.inscripcion_id === est.inscripcion_id
        );
        return (
          <TableRow key={est.inscripcion_id}>
            <TableCell>{index + 1}</TableCell>
            <TableCell>
              <div>
                <p className="font-medium">
                  {est.estudiante.apellido},
                  {" "}{est.estudiante.nombre}
                </p>
                <p className="text-xs text-muted-foreground">
                  {est.estudiante.email}
                </p>
              </div>
            </TableCell>
            <TableCell>
              <Select

```

```

        value={registro?.estado || "presente"}
        onValueChange={(v) =>
            actualizarEstado(est.inscripcion_id, v)
        }
    >
    <SelectTrigger className="w-[140px]">
        <SelectValue />
    </SelectTrigger>
    <SelectContent>
        {ESTADOS_ASISTENCIA.map((ea) => (
            <SelectItem key={ea.value} value={ea.value}>
                {ea.label}
            </SelectItem>
        ))}
    </SelectContent>
</Select>
</TableCell>
<TableCell>
    <Input
        placeholder="Opcional..."
        value={registro?.observaciones || ""}
        onChange={(e) =>
            actualizarObservacion(
                est.inscripcion_id,
                e.target.value
            )
        }
        className="max-w-[200px]"
    />
</TableCell>
</TableRow>
    );
    })}
</TableBody>
</Table>

{/* Boton Guardar */}
<div className="flex justify-end mt-6">
    <Button
        onClick={guardarAsistencia}
        disabled={guardando}
        size="lg"
    >
        <Save className="h-4 w-4 mr-2" />
        {guardando
            ? "Guardando..."
            : "Guardar Asistencia"
        }
    </Button>
</div>
</CardContent>
</Card>
)}

```

```
    </div>  
  );  
}
```

TIP: Patron de estado local con sync posterior

Los registros[] se manipulan localmente (en estado React) mientras el docente selecciona estados. Solo cuando hace click en 'Guardar Asistencia' se envian al servidor. Esto evita N peticiones durante la edicion y permite cancelar sin efectos secundarios.

Paso 6: Componente Historial de Asistencias

El historial permite al docente revisar las asistencias pasadas con filtro por fecha. Además, puede editar un registro individual si cometió un error.

TypeScript - components/dashboard/docente/historial-asistencias.tsx

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { History, Filter } from "lucide-react";

import { Button } from "@components/ui/button";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";

import { ESTADOS_ASISTENCIA } from "@lib/validations/asistencia";

interface Props {
  materiaId: string;
}

interface RegistroHistorial {
  id: string;
  fecha: string;
  estado: string;
  observaciones: string | null;
  inscripciones: {
    id: string;
    usuarios_perfil: {
      nombre: string;
      apellido: string;
    };
  };
};

export default function HistorialAsistencias({ materiaId }: Props) {
  const [historial, setHistorial] = useState<RegistroHistorial[]>([]);
  const [fechas, setFechas] = useState<string[]>([]);
```



```

const [filtroFecha, setFiltroFecha] = useState<string>("todas");
const [filtroEstado, setFiltroEstado] = useState<string>("todos");
const [cargando, setCargando] = useState(true);

useEffect(() => {
  async function cargar() {
    try {
      const params = new URLSearchParams({
        materia_id: materiaId,
        modo: "historial",
      });
      const res = await fetch(`/api/docente/asistencias?${params}`);
      if (res.ok) {
        const data = await res.json();
        setHistorial(data.historial);
        setFechas(data.fechas);
      }
    } catch {
      toast.error("Error al cargar historial");
    } finally {
      setCargando(false);
    }
  }
  if (materiaId) cargar();
}, [materiaId]);

// Filtrar
const registrosFiltrados = historial.filter((r) => {
  const matchFecha = filtroFecha === "todas" || r.fecha === filtroFecha;
  const matchEstado = filtroEstado === "todos" || r.estado === filtroEstado;
  return matchFecha && matchEstado;
});

// Edicion rapida de estado
async function editarEstado(id: string, nuevoEstado: string) {
  try {
    const res = await fetch(`/api/docente/asistencias/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ estado: nuevoEstado }),
    });

    if (res.ok) {
      setHistorial((prev) =>
        prev.map((r) =>
          r.id === id ? { ...r, estado: nuevoEstado } : r
        )
      );
      toast.success("Estado actualizado");
    } else {
      const err = await res.json();
      toast.error(err.error);
    }
  }
}

```

```

    }
  } catch {
    toast.error("Error al actualizar");
  }
}

if (cargando) {
  return <p className="text-muted-foreground">Cargando historial...</p>;
}

return (
  <Card>
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <History className="h-5 w-5" />
        Historial de Asistencias ({historial.length} registros)
      </CardTitle>
    </CardHeader>
    <CardContent>
      {/* Filtros */}
      <div className="flex gap-4 mb-4">
        <div className="flex items-center gap-2">
          <Filter className="h-4 w-4 text-muted-foreground" />
          <Select value={filtroFecha} onChange={setFiltroFecha}>
            <SelectTrigger className="w-[180px]">
              <SelectValue placeholder="Filtrar por fecha" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="todas">Todas las fechas</SelectItem>
              {fechas.map((f) => (
                <SelectItem key={f} value={f}>{f}</SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>

        <Select value={filtroEstado} onChange={setFiltroEstado}>
          <SelectTrigger className="w-[160px]">
            <SelectValue placeholder="Filtrar por estado" />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="todos">Todos</SelectItem>
            {ESTADOS_ASISTENCIA.map((ea) => (
              <SelectItem key={ea.value} value={ea.value}>
                {ea.label}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>

      {/* Tabla */}

```

```

<Table>
  <TableHeader>
    <TableRow>
      <TableHead>Fecha</TableHead>
      <TableHead>Estudiante</TableHead>
      <TableHead>Estado</TableHead>
      <TableHead>Observaciones</TableHead>
    </TableRow>
  </TableHeader>
  <TableBody>
    {registrosFiltrados.map((r) => (
      <TableRow key={r.id}>
        <TableCell className="font-mono text-sm">
          {r.fecha}
        </TableCell>
        <TableCell>
          {r.inscripciones.usuarios_perfil.apellido},
          {" "}{r.inscripciones.usuarios_perfil.nombre}
        </TableCell>
        <TableCell>
          <Select
            value={r.estado}
            onChange={(v) => editarEstado(r.id, v)}
          >
            <SelectTrigger className="w-[140px]">
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              {ESTADOS_ASISTENCIA.map((ea) => (
                <SelectItem key={ea.value} value={ea.value}>
                  {ea.label}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </TableCell>
        <TableCell className="text-sm text-muted-foreground">
          {r.observaciones || "-"}
        </TableCell>
      </TableRow>
    ))}
    {registrosFiltrados.length === 0 && (
      <TableRow>
        <TableCell
          colspan={4}
          className="text-center text-muted-foreground py-8"
        >
          Sin registros para los filtros seleccionados
        </TableCell>
      </TableRow>
    )}
  </TableBody>
</Table>

```

```
        </CardContent>  
      </Card>  
    );  
  }
```

ADVERTENCIA: Edicion inline en el historial

El Select de estado en el historial ejecuta un PUT inmediato al cambiar valor (actualizacion optimista). El estado local se actualiza al instante y si el servidor falla, se muestra un toast de error. No hay boton 'Guardar' aqui; es edicion rapida.

Paso 7: Actualizar Pagina Docente de Asistencias

Reemplaza la pagina placeholder con una implementacion que combina los dos componentes usando Tabs (pestanas) para alternar entre 'Pasar Lista' e 'Historial'.

TypeScript - app/(dashboard)/docente/asistencias/page.tsx (reemplazar)

```
"use client";

import { useState } from "react";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import { ClipboardList, History } from "lucide-react";

import PasarLista from "@components/dashboard/docente/pasar-lista";
import HistorialAsistencias from "@components/dashboard/docente/historial-asistencias";

import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { Label } from "@components/ui/label";
import { useEffect } from "react";

interface Materia {
  id: string;
  nombre: string;
}

export default function AsistenciasDocentePage() {
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [materiaHistorial, setMateriaHistorial] = useState("");

  useEffect(() => {
    async function cargar() {
      const res = await fetch("/api/docente/materias");
      if (res.ok) setMaterias(await res.json());
    }
    cargar();
  }, []);

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">Asistencias</h1>

      <Tabs defaultValue="pasar-lista">
        <TabsList>
          <TabsTrigger value="pasar-lista">
            <ClipboardList className="h-4 w-4 mr-2" />

```

```

        Pasar Lista
      </TabsTrigger>
      <TabsTrigger value="historial">
        <History className="h-4 w-4 mr-2" />
        Historial
      </TabsTrigger>
    </TabsList>

    <TabsContent value="pasar-lista" className="mt-4">
      <PasarLista />
    </TabsContent>

    <TabsContent value="historial" className="mt-4">
      <div className="mb-4">
        <Label>Seleccionar materia para ver historial</Label>
        <Select
          value={materiaHistorial}
          onChange={setMateriaHistorial}
        >
          <SelectTrigger className="w-[300px] mt-2">
            <SelectValue placeholder="Seleccionar materia" />
          </SelectTrigger>
          <SelectContent>
            {materias.map((m) => (
              <SelectItem key={m.id} value={m.id}>
                {m.nombre}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>

      {materiaHistorial && (
        <HistorialAsistencias materiaId={materiaHistorial} />
      )}
    </TabsContent>
  </Tabs>
</div>
);
}

```

NOTA: Tabs de shadcn/ui

El componente Tabs viene pre-instalado con shadcn/ui. Si no lo tienes, ejecuta: `npx shadcn@latest add tabs`. Las pestañas permiten alternar sin recargar la página, manteniendo el estado de cada componente hijo.

Verificacion Final

Verifica cada punto antes de continuar con la Entrega 9B:

- ✓ lib/validations/asistencia.ts exporta asistenciaRegistroSchema, asistenciaMasivaSchema, ESTADOS_ASISTENCIA.
- ✓ GET /api/docente/asistencias?materia_id=X&fecha=Y&modo=lista devuelve estudiantes con asistencias pre-cargadas.
- ✓ GET /api/docente/asistencias?materia_id=X&modo=historial devuelve historial completo con fechas unicas.
- ✓ POST /api/docente/asistencias crea/actualiza registros masivos con UPSERT.
- ✓ PUT /api/docente/asistencias/[id] permite editar un registro individual.
- ✓ El componente PasarLista muestra la tabla de estudiantes con Selects de estado y botones 'Todos Presente/Ausente'.
- ✓ Los contadores de resumen (badges) se actualizan en tiempo real al cambiar estados.
- ✓ El segundo envio en la misma fecha actualiza en vez de crear duplicados (UPSERT).
- ✓ El historial carga correctamente con filtros por fecha y estado.
- ✓ La edicion inline en el historial ejecuta PUT inmediato y actualiza el estado local.
- ✓ La pagina usa Tabs para alternar entre 'Pasar Lista' e 'Historial'.

Ejercicio Propuesto

1. Tarea 1: Agrega un boton 'Exportar CSV' al historial que genere un archivo descargable con columnas: Fecha, Estudiante, Estado, Observaciones. Pista: usa Blob + URL.createObjectURL().
2. Tarea 2: Modifica el componente PasarLista para que muestre un indicador visual (icono o color de fondo) cuando un registro ya existia previamente (es decir, el docente ya paso lista ese dia y esta editando).
3. Tarea 3: Agrega un resumen estadistico al historial: porcentaje de asistencia por estudiante (presente + tardanza / total de sesiones). Muestra un badge rojo si el porcentaje es menor al 70%.
4. Tarea 4: Implementa un selector de rango de fechas en el historial (fecha_desde y fecha_hasta) que filtre los registros en el frontend. Pista: modifica los filtros existentes.

Proximo: Entrega 9B

En la Entrega 9B implementaremos las vistas de consulta de asistencias para estudiantes y admin: API Routes de solo lectura, componente de 'mis asistencias' para el estudiante con resumen estadístico, vista admin con panorama global, y tarjetas de estadísticas para el dashboard.

Que vamos a construir	Archivos nuevos
API Route: GET asistencias (estudiante)	app/api/estudiante/asistencias/route.ts
API Route: GET asistencias global (admin)	app/api/admin/asistencias/route.ts
Componente: mis asistencias (estudiante)	components/dashboard/estudiante/mis-asistencias.tsx
Componente: resumen global (admin)	components/dashboard/admin/asistencias-resumen.tsx
Tarjetas de estadísticas de asistencia	Actualizar /admin/stats y /docente/stats